Amrita Vishwa Vidyapeetham
Department of Artificial Intelligence

Database Management Systems — Course Project Report

Bus Factor Tracker

Estimating Open-Source Maintainer Burnout and Bus Factor Risk from Real GitHub Data

---

Submitted by

Dhakshin	CB.SC.U4AIE24313
Rohith Ravi	CB.SC.U4AIE24350
Sanjay Kumar S	CB.SC.U4AIE24354
Akhilan	CB.SC.U4AIE24362

Semester V • October 2026

Abstract

Open-source software forms the base layer of almost every modern application, yet much of it is maintained by a handful of volunteers. When such a maintainer burns out or leaves, every project that depends on their work is exposed, often with no warning. The *bus factor* of a project, the number of people who would have to disappear before it stalls, is a simple measure of this risk.

This report presents the **Bus Factor Tracker**, a database-centred system that estimates bus factor and maintainer-burnout risk for real GitHub repositories from public data only. A Python sync engine pulls repository metadata, per-contributor weekly commit counts, issues, pull requests and releases from the GitHub REST API into a MySQL 8.0 database of ten relations in Boyce–Codd normal form, writing each repository inside a single ACID transaction. Automation bots are identified and excluded, and the bus factor is computed in SQL with common table expressions and window functions over the last twelve months of commits, falling back to the all-time value for repositories with too little recent work. Burnout signals (no recent commits, falling commit activity, an inactive key contributor and slow issue responses) raise a base risk level derived from the bus factor. Because the weekly data goes back years, the first sync rebuilds a year of monthly snapshots, so trends are available immediately. A view, a stored function, a stored procedure and a trigger expose the latest risk, detect declining repositories and record every risk increase, comparing only snapshots produced by the same version of the scoring method. A FastAPI service and a React dashboard present the results with a plain-English explanation of each score.

Applied to 42 widely used repositories (17,306 contributors, 273,603 contributor-weeks of commits, 16,490 pull requests), the system found that 21 repositories (50%) depend on a single person for at least half of their recent commits. The rebuilt history contains 16 risk increases, all recorded by the trigger, including a fall in the bus factor of `httpx` from 5 to 1 within six months. Excluding bots changed the picture for large projects: Kubernetes' all-time bus factor rose from 18 to 44 once its CI robot was removed. All read endpoints of the API respond in about 10 ms or less.

Keywords: bus factor, truck factor, open-source sustainability, maintainer burnout, relational database, MySQL, window functions, triggers, time-series snapshots, GitHub REST API.

Contents

1 Introduction **5**

1.1 Background . . . 5
1.2 The bus factor . . . 5
1.3 Problem statement . . . 5
1.4 Objectives . . . 6
1.5 Scope . . . 6
1.6 Related work . . . 6
1.7 Organisation of the report . . . 7

2 Methodology **8**

2.1 System architecture . . . 8
2.2 Data collection . . . 8
2.3 Database design . . . 9
2.3.1 Entity–relationship model . . . 9
2.3.2 Relational schema . . . 12
2.3.3 Normalisation . . . 12
2.3.4 Indexes . . . 13
2.4 Transaction design . . . 13
2.5 Computing the measurements in SQL . . . 14
2.5.1 Bus factor . . . 14
2.5.2 Median issue-response time . . . 15
2.5.3 Burnout signals . . . 15
2.6 Risk scoring . . . 15
2.7 History back-fill and scoring versions . . . 17
2.8 Database objects . . . 17
2.9 Application layer . . . 18

3 Results and Discussion **19**

3.1 Experimental setup . . . 19
3.2 Data collected . . . 19
3.3 Risk distribution . . . 20
3.4 Effect of excluding bots and measuring recent work . . . 21
3.5 Trends and alerts . . . 22
3.6 Case studies and validation . . . 23

3.7 Performance 24
3.8 Per-repository results 25
3.9 Discussion of limitations 26
3.10 Challenges and lessons learned 27

4 Conclusion and Future Work **28**
4.1 Conclusion 28
4.2 Future work 28

References **29**

List of Figures

2.1 System architecture and data flow. 8
2.2 Entity–relationship diagram. Underlined attributes form the primary key; italic attributes are foreign keys. Solid lines are mandatory relationships, dashed lines optional ones (a deleted GitHub account leaves the author unknown). Contributor_Weekly_Commits breaks each repository–contributor pair down by week; its keys reference Repository and Contributor. Risk_Alert also references Repository (not drawn). 11

3.1 Risk level of the 42 repositories (latest snapshot). 20
3.2 Effective bus factor versus top-contributor share for the 42 repositories. 21
3.3 Bus factor over time for `encode/httpx`: monthly back-filled snapshots from November 2025 and the current one. 22
3.4 Dashboard: summary counts per risk level and the recent risk increases recorded by the trigger, dated by when the risk rose. 24
3.5 Repository detail for `encode/httpx`: recent and all-time bus factor, the reasons for the risk level, and the burnout signals. 24

List of Tables

2.1 GitHub REST API endpoints used per repository. 8
2.2 Relations, keys and purpose. 12
2.3 Normal forms as applied to the schema. 13
2.4 Risk levels and burnout thresholds. 16

3.1 Rows stored per relation. 19
3.2 Risk levels with average effective bus factor and top-contributor share. 20
3.3 Average effective bus factor by primary language (languages with ≥ 2 repositories). 21
3.4 Risk increases recorded by `trg_snapshot_risk_alert` (scoring version 2). 23
3.5 Median API latency over 20 requests. 25
3.6 Latest snapshot per repository (bots excluded). BF 12m = bus factor over the last 12 months ("–" = fewer than 10 recent commits); BF all = all-time bus factor; Top = top contributor's share of commits in the window the rating uses; ΔAct = commit-activity change ("–" = baseline below one commit per week); Inact. = days since that contributor's last commit. 25

Chapter 1

Introduction

1.1 Background

Open-source libraries are reused at enormous scale: a single web application can pull in hundreds of transitive dependencies, each maintained by people the application's developers have never met. Studies of the open-source ecosystem describe this as critical public infrastructure that is largely maintained without pay [1, 2]. The consequences of a single maintainer's decision became widely visible in March 2016, when the removal of the eleven-line `left-pad` package from the npm registry broke the builds of thousands of downstream projects [3].

Less dramatic versions of the same failure happen continually. Maintainers lose interest, change jobs or simply burn out, and projects that looked healthy from the outside stop receiving fixes. Research on failed open-source projects lists lack of maintainer time and interest among the most common reasons a project is abandoned [4], and toxic or demanding interactions with users are a documented source of maintainer stress [5].

1.2 The bus factor

The *bus factor* (also called the *truck factor*) of a software project is the minimum number of developers who would have to leave the project ("be hit by a bus") before it can no longer be maintained. A bus factor of 1 means that a single person holds knowledge or responsibility the project cannot function without. Several algorithms have been proposed to estimate it from version-control history, including file-authorship approaches [6] and commit-based approaches applied directly to Git repositories [7]. Practitioners report that the metric is useful but that raw estimates need context to be acted on [8].

1.3 Problem statement

Developers choosing a dependency, and organisations funding open-source work, have no simple way to see which projects are one person away from abandonment. Existing tools either show raw activity graphs for a single repository, focus on security practices, or require a heavyweight analytics platform (Section 1.6). The problem addressed in this project is:

Design and build a database-backed system that estimates the bus factor and maintainer-burnout risk of real GitHub repositories from public data, explains each estimate, and tracks how the risk changes over time.

1.4 Objectives

1. Collect real repository, contributor, issue, pull-request and release data from the GitHub REST API — no simulated data.
2. Design a normalised relational schema (BCNF) with full referential integrity, suitable for both current-state queries and time-series history.
3. Compute the bus factor, median issue-response time and burnout signals inside the database using SQL (CTEs and window functions).
4. Use DBMS features where they fit the problem: transactions for consistent syncs, a view, a stored function, a stored procedure and a trigger.
5. Combine the measurements into an explainable risk level per repository, and record every increase in risk.
6. Present the results through a REST API and an interactive web dashboard.

1.5 Scope

The system tracks a configurable list of public GitHub repositories (42 in the evaluation, chosen to mix large, well-staffed projects with small single-maintainer utilities and one archived project). It relies on the data GitHub exposes through its public API with a personal access token. It does not analyse source code, mailing lists or chat channels, and it does not attempt per-file authorship analysis.

1.6 Related work

Truck-factor research. Avelino et al. [6] estimate truck factors from file authorship (degree-of-authorship) and validate the estimates with developers of popular GitHub projects. Cosentino et al. [7] compute bus-factor indicators for Git repositories. Jabrayilzade et al. [8] study how the bus factor is used in industry. These works analyse repositories at a point in time; they do not maintain a continuously updated history or alert on change.

Community-health tooling. The CHAOSS project [9] defines a broad catalogue of open-source community-health metrics and provides analytics platforms to compute them. The OpenSSF Scorecard [10] assesses security-related practices of a repository, including whether it has been recently active. GitHub's own repository Insights pages show activity graphs for one repository at a time.

This project. The Bus Factor Tracker sits between these: it is a lightweight, single-purpose system that (i) combines commit concentration with burnout signals into one explainable risk level, (ii) ranks many repositories side by side, and (iii) keeps an append-only history so that declines and risk increases are detected inside the database itself.

1.7 Organisation of the report

Chapter 2 describes the system architecture, data collection, database design, SQL techniques and risk-scoring method. Chapter 3 presents the results on 42 real repositories and discusses them, including performance, validation cases and limitations. Chapter 4 concludes and outlines future work.

Chapter 2

Methodology

2.1 System architecture

The system is split into four modules connected by the database (Figure 2.1). Collecting data is slow (tens of seconds of API calls per repository) while reading results must be fast, so the two are deliberately separated: the sync engine writes, and the API almost only reads.

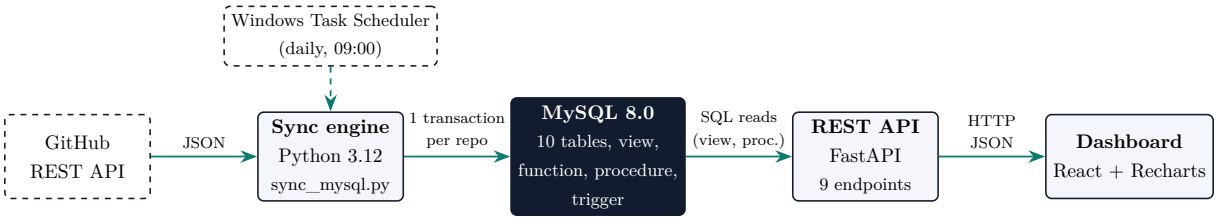


Figure 2.1: System architecture and data flow.

2.2 Data collection

The sync engine (`scripts/sync_mysql.py`) reads the list of tracked repositories from `repos.txt` and, for each repository, calls the GitHub REST API endpoints in Table 2.1 [11].

Table 2.1: GitHub REST API endpoints used per repository.

Endpoint	Data obtained
`/repos/{o}/{r}`	Language, stars, forks, creation date, archived flag
`/repos/{o}/{r}/stats/contributors`	For each of the top 100 contributors: account type and commits, additions and deletions per week since the repository began
`/repos/{o}/{r}/stats/commit_activity`	Total commits per week for the last 52 weeks
`/repos/{o}/{r}/issues`	Issues (state, opened/closed time, author); pull requests filtered out
`/repos/{o}/{r}/issues/{n}/comments`	Comments on an issue, giving the first-response time
`/repos/{o}/{r}/pulls`	Pull requests (state, opened/merged time, author)
`/repos/{o}/{r}/releases`	Releases (tag, publish time, author)

Several design choices keep the sync within GitHub's limit of 5,000 authenticated requests per hour and make it robust:

- **Pre-aggregated statistics.** Contributor and weekly-activity statistics come from GitHub's `stats` endpoints, one call each, instead of one call per commit.
- **Incremental sync.** The repository's `last_synced_at` timestamp is read before each sync; issues are requested with GitHub's `since` parameter, and pull-request and release pagination stops as soon as it reaches items older than the last sync.
- **Capped work.** Pagination is capped at three pages, and the first-response lookup is limited to the 30 most recently updated issues with comments; those lookups run concurrently (10 threads).
- **Lazy statistics.** GitHub computes `stats` endpoints in the background and answers `202 Accepted` until they are ready. The engine retries with capped exponential back-off (2, 4, 8, 10, 10 s). If `commit_activity` never becomes ready, weekly totals are derived from the per-contributor weekly counts instead.
- **Network resilience.** Dropped connections, truncated responses and time-outs are retried up to three times per request.
- **No network I/O inside a transaction.** All API calls for a repository complete before its database transaction begins, so slow responses never hold locks.

Bot detection. Many projects run automation accounts that commit, merge or comment on everyone's behalf. GitHub marks GitHub Apps with the account type `Bot` (e.g. `dependabot[bot]`), but many CI bots are ordinary user accounts (`k8s-ci-robot`, `pytorchmergebot`, `tensorflower-gardener`). An account is therefore treated as a bot if its type is `Bot` or its login ends in `[bot]`, `bot` or `-gardener`. Checked against all 17,306 contributors in the data set, the rule flags 52 accounts; the few that may be people with "bot"-like names have 60 commits or fewer, too few to affect any bus factor. Bots are kept in the database (flag `contributor.is_bot`) but excluded from every calculation.

First response to an issue. The first-response time is the time of the first comment by someone other than the issue's author who is not a bot. Taking simply the first comment would measure bots: on Go, Kubernetes and PyTorch, a bot comments on every new issue within seconds.

A Windows Task Scheduler job runs the sync daily, so that each repository gains one snapshot per day.

2.3 Database design

2.3.1 Entity–relationship model

The conceptual model (Figure 2.2) follows Chen's entity–relationship approach [12]. Repository and Contributor are the two strong entities. Their many-to-many relationship carries its

own attributes (commit counts and first/last commit dates), so it is modelled as the associative entity Repo_Contributor. Each repository–contributor pair is further broken down by week in Contributor_Weekly_Commits, a weak entity identified by the pair and the week; it is what allows the bus factor to be computed for any time window and as of any past date. Weekly_Activity is a weak entity identified by its repository and week. Snapshots and alerts form an append-only history.

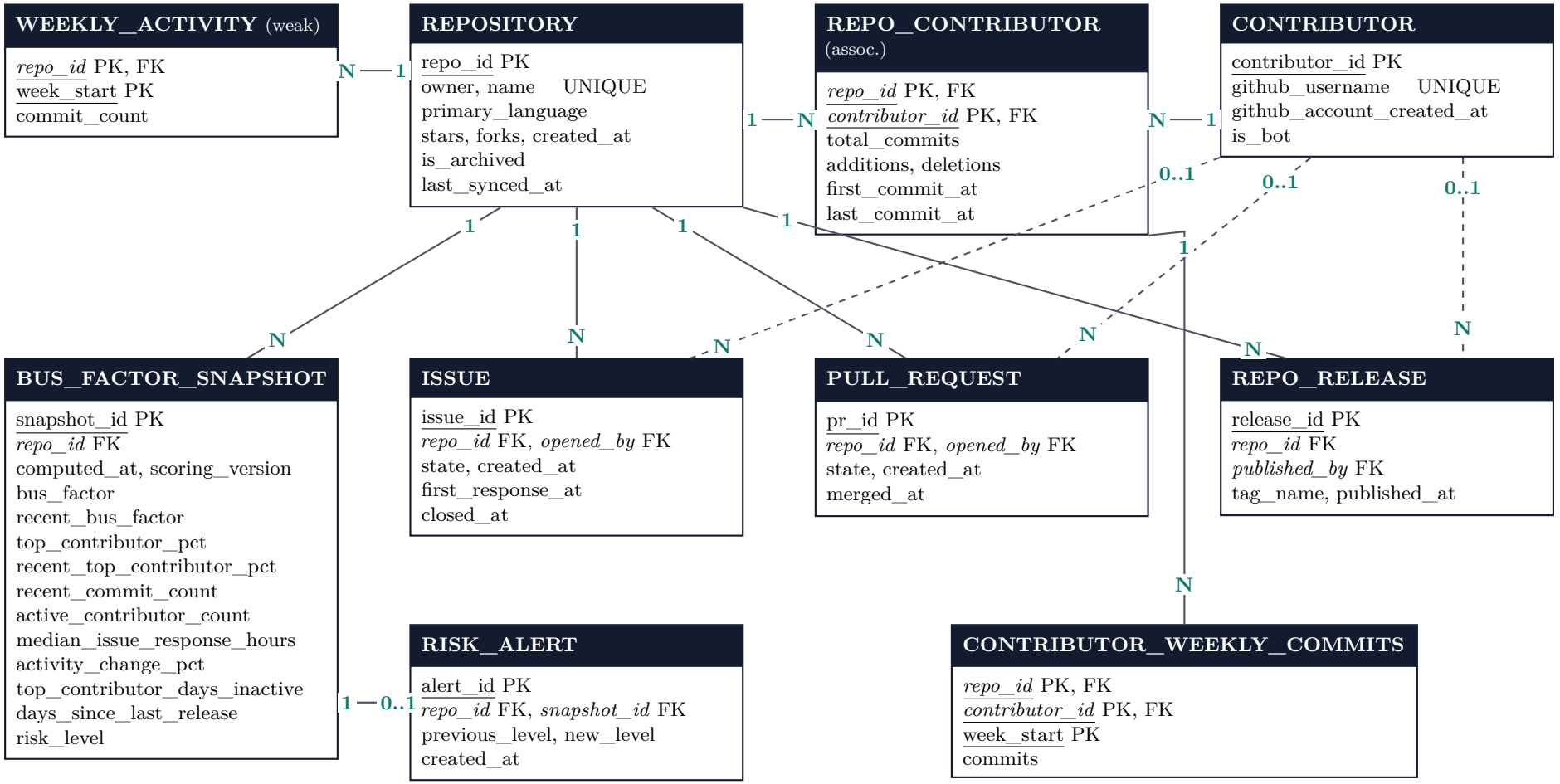


Figure 2.2: Entity–relationship diagram. Underlined attributes form the primary key; italic attributes are foreign keys. Solid lines are mandatory relationships, dashed lines optional ones (a deleted GitHub account leaves the author unknown). Contributor_Weekly_Commits breaks each repository–contributor pair down by week; its keys reference Repository and Contributor. Risk_Alert also references Repository (not drawn).

2.3.2 Relational schema

The ER model maps to ten relations (Table 2.2). GitHub's own identifiers are used as primary keys for issues, pull requests and releases (`BIGINT`), which makes re-syncing idempotent and guarantees that keys never collide across repositories. `release` is a reserved word in MySQL, hence `repo_release`.

Table 2.2: Relations, keys and purpose.

Relation	Primary key / foreign keys	Purpose
`repository`	repo_id; UNIQUE(owner, name)	One row per tracked repository
`contributor`	contributor_id; UNIQUE(github_username)	People and bots (`is_bot`), shared across repositories
`repo_contributor`	(repo_id, contributor_id)	Associative entity: commits per person per repository
`contributor_weekly_commits`	(repo_id, contributor_id, week_start)	Weak entity: commits per person per week
`issue`	issue_id; FK repo_id, opened_by	Issues with first-response time
`pull_request`	pr_id; FK repo_id, opened_by	Pull requests
`repo_release`	release_id; FK repo_id, published_by	Releases
`weekly_activity`	(repo_id, week_start)	Weak entity: repository commits per week
`bus_factor_snapshot`	snapshot_id; FK repo_id	Append-only, versioned time series of computed risk
`risk_alert`	alert_id; FK repo_id, snapshot_id	Written by a trigger when risk increases

Referential integrity. Every child relation references `repository(repo_id)` with `ON DELETE CASCADE`. Removing a repository from the dashboard is therefore a single `DELETE` statement, and the DBMS removes its contributor statistics, weekly commits, issues, pull requests, releases, snapshots and alerts in the same operation. Contributors are shared between repositories, so they are deliberately not cascaded.

2.3.3 Normalisation

All relations are in Boyce–Codd normal form: for every non-trivial functional dependency $X \to Y$, X is a superkey [13–15]. Table 2.3 shows how each normal form is reflected in the design.

Table 2.3: Normal forms as applied to the schema.

Normal form	Application
1NF	Contributors of a repository are rows in `repo_contributor`, not a comma-separated list; weekly commits are one row per week, not an array.
2NF	In `contributor_weekly_commits`, `commits` depends on the whole key (repository, contributor, week); `github_username` and `is_bot` depend on `contributor_id` alone and therefore live in `contributor`.
3NF	`risk_level` is derived from the bus factor and burnout signals and changes on every sync, so it is stored on the snapshot that produced it, never on `repository`.
BCNF	`repository` has two candidate keys, `repo_id` and (`owner`, `name`); both determine all other attributes. The same holds for `contributor`.

Deliberate redundancy. Two derived values are stored on purpose. The `total_commits` column of `repo_contributor` is GitHub's reported total for a contributor, kept so the contributor list can be read without aggregating weekly rows. `bus_factor_snapshot` stores values that could in principle be recomputed: a snapshot records what the measurements *were* at `computed_at`, under the scoring method of that time.

2.3.4 Indexes

Besides primary keys and unique constraints, secondary indexes support the most frequent access paths: `(repo_id, week_start)` on weekly commits for windowed bus-factor queries, `(repo_id, scoring_version, computed_at)` on snapshots for "latest snapshot", trend and trigger queries, `repo_id` on `repo_contributor` and `issue`, and `created_at` on `risk_alert` for the newest-first alert feed.

2.4 Transaction design

Each repository is synchronised in one transaction (Listing 2.1), following the atomicity and durability guarantees of ACID transactions [16]. If any step fails, such as a network error during the preceding fetch, a malformed row or a lost database connection, the whole repository rolls back. The previous snapshot then remains the latest one, so the dashboard never shows a score computed from half-updated data. Every write except the snapshots is an `INSERT ... ON DUPLICATE KEY UPDATE`, making repeated syncs idempotent.

Listing 2.1: Per-repository transaction (simplified from `sync_repo`).

```
cur = conn.cursor() # conn.autocommit = False
try:
    repo_id = upsert_repo(cur, owner, repo, meta)
    for entry in contributor_stats:
        cid = upsert_contributor(cur, login, account_type) # sets is_bot
        upsert_repo_contributor(cur, repo_id, cid, ...)
        upsert_contributor_weeks(cur, repo_id, cid, weeks)
    upsert_weekly_activity(cur, repo_id, commit_activity)
    for issue in issues: upsert_issue(cur, repo_id, ...)
```

```
    for pr in pull_requests: upsert_pull_request(cur, repo_id, ...)
    for release in releases: upsert_release(cur, repo_id, ...)
    compute_and_insert_snapshot(cur, repo_id) # back-fill if needed; fires the trigger
    conn.commit()
except Exception:
    conn.rollback() # nothing for this repository is kept
    raise
```

2.5 Computing the measurements in SQL

All measurements are computed by one function, `compute_metrics(repo, as_of)`, that takes the date the snapshot describes as a parameter. Today's snapshot and the back-filled history (Section 2.7) are therefore produced by exactly the same queries.

2.5.1 Bus factor

The bus factor of a time window is the smallest number of contributors who together authored at least 50% of the commits in that window, bots excluded. Listing 2.2 computes it in a single pass: commits per person in the window are summed from `contributor_weekly_commits`, people are ranked by commits (ties broken by id so that the running total is strictly cumulative), and the first rank at which the running total reaches half of all commits is the bus factor.

Listing 2.2: , bots excluded.]Bus factor over a window of weeks (after, until], bots excluded.

```
WITH per_person AS (
    SELECT w.contributor_id, SUM(w.commits) AS commits
    FROM contributor_weekly_commits w
    JOIN contributor c ON c.contributor_id = w.contributor_id
    WHERE w.repo_id = %s AND c.is_bot = FALSE
      AND w.week_start > %s AND w.week_start <= %s
    GROUP BY w.contributor_id
),
ranked AS (
    SELECT contributor_id, commits,
           SUM(commits) OVER () AS total,
           SUM(commits) OVER (ORDER BY commits DESC, contributor_id) AS running_total,
           ROW_NUMBER() OVER (ORDER BY commits DESC, contributor_id) AS contributor_rank,
           COUNT(*) OVER () AS people
    FROM per_person
)
SELECT MIN(CASE WHEN running_total >= 0.5 * total THEN contributor_rank END) AS bus_factor,
       MAX(CASE WHEN contributor_rank = 1
                THEN ROUND(100 * commits / total, 1) END) AS top_pct,
       MAX(total) AS commits, MAX(people) AS people
FROM ranked;
```

The query is run twice per snapshot:

- **All-time:** every week up to the snapshot date.

- **Last 12 months:** the 52 weeks before the snapshot date.

The risk level is based on the *recent* bus factor, which reflects who maintains the project now, unless the repository had fewer than 10 non-bot commits in those 12 months. In that case there is too little recent work to measure, and the all-time value is used instead. The view exposes the value in use as `effective_bus_factor`.

2.5.2 Median issue-response time

`PERCENTILE_CONT` is only available from MySQL 8.0.28 [17], so the median uses a portable window-function form: response times of issues opened in the 12 months before the snapshot date are numbered with `ROW_NUMBER()`, counted with `COUNT(*) OVER ()`, and the middle one or two are averaged. At least five answered issues are required; otherwise the median is reported as unknown.

2.5.3 Burnout signals

No recent work. Zero non-bot commits in the last 12 months.

Commit-activity change. Non-bot commits per week in the most recent 12 weeks compared with the 40 weeks before, computed by conditional aggregation over `contributor_weekly_commits`:

$$\Delta_{\text{act}} = 100 \times \frac{\overline{c}_{\text{last 12 wk}} - \overline{c}_{\text{prev 40 wk}}}{\overline{c}_{\text{prev 40 wk}}} \ \%.$$

Weeks without commits have no row, so sums are divided by the number of weeks (12 and 40) rather than averaged over rows. The change is only reported when the earlier period averaged at least one commit per week; below that a percentage is noise (one repository showed +10,900% under the first version of the method).

Key-contributor inactivity. Days since the last commit week of the top contributor in the window the rating is based on.

Slow issue response. The median issue-response time described above.

Days since the latest release is also stored, but only as context: many healthy libraries never publish GitHub releases.

2.6 Risk scoring

Algorithm 1 combines the measurements. The effective bus factor sets a base level; an archived repository is Critical outright; and each independent burnout signal raises the level by one, capped at Critical. The function returns the level together with a list of plain-English reasons. The same function is used by the sync engine (to store the level) and by the API (to explain it), so the explanation always matches the stored score.

Algorithm 1 Risk assessment (`assess_risk`)

Require: all-time bus factor b, recent bus factor b_r (or none), top-contributor share s (%) of the window in use, recent commits n_r, median response m (h), activity change a (%), key-contributor inactivity d (days), archived flag

Ensure: risk level $L \in \{0, 1, 2, 3\}$ (Healthy, Medium, High, Critical) and reasons R

$e \leftarrow b_r$ **if** b_r is defined **else** b ▷ effective bus factor
$L \leftarrow 3$ **if** $e = 1$ **else** 2 **if** $e = 2$ **else** 1 **if** $e \leq 4$ **else** 0
$R \leftarrow$ [explanation of e, s and, if different, b]
if archived **then**
    $L \leftarrow 3$; add "repository is archived" to R
end if
if $n_r = 0$ **then**
    $L \leftarrow L + 1$; add "no commits in the last 12 months" to R
end if
if $m > 720$ **then**
    $L \leftarrow L + 1$; add "slow issue responses" to R
end if
if $a \leq -50$ **then**
    $L \leftarrow L + 1$; add "commit activity down" to R
end if
if $d > 180$ **and** $s \geq 25$ **then**
    $L \leftarrow L + 1$; add "top contributor inactive" to R
end if
return $(\min(L, 3),\ R)$

Table 2.4: Risk levels and burnout thresholds.

Component	**Condition**	**Effect**
Effective bus factor	1 / 2 / 3–4 / ≥ 5	Critical / High / Medium / Healthy
Archived repository	`is_archived = TRUE`	Critical
No recent work	no non-bot commits in the last 12 months	+1 level
Slow issue responses	median first human reply > 30 days (≥ 5 issues)	+1 level
Activity decline	last 12 weeks $\geq 50\%$ below previous 40 (baseline ≥ 1/week)	+1 level
Key contributor inactive	no commit > 180 days *and* share $\geq 25\%$	+1 level
Release recency	days since last release	shown only

The 25% share condition on the inactivity signal was added after testing on real data (Section 3.6).

2.7 History back-fill and scoring versions

Back-fill. A daily sync only starts producing a trend after weeks of snapshots. But `contributor_weekly_commits` holds each contributor's commits since the repository began, so the past can be measured directly. The first time a repository is scored with the current method, `backfill_history` calls `compute_metrics` for the first day of each of the previous 12 months, oldest first, and inserts one snapshot per month with `computed_at` set to that date. Issue responses and releases are filtered to what was known at each date. Only the archived flag cannot be reconstructed, since GitHub reports only its current value, so back-filled snapshots treat the repository as active.

Scoring versions. Changing how risk is computed creates a database problem: comparing a snapshot from the old method with one from the new method would report a change in the project that is really a change in the method. Each snapshot therefore stores `scoring_version` (1 = original all-time method, 2 = bots excluded and recency-based). The view prefers the newest version, and the procedure, the trigger, the trend endpoint and the alerts feed only ever compare snapshots of the same version.

2.8 Database objects

View. `v_repo_latest_snapshot` joins every repository to its newest snapshot (a `LEFT JOIN`, so repositories without a snapshot still appear), ordering by `scoring_version`, then `computed_at`, then `snapshot_id`. Ordering by time rather than insertion matters because the back-fill inserts past-dated rows. The view also derives `effective_bus_factor` and `effective_top_contributor_pct` with `COALESCE`. All API endpoints that need "current state" read from it.

Stored function. `fn_risk_rank(level)` maps Healthy, Medium, High and Critical to 1–4. The `ENUM` stores the levels in the opposite order, so this function is the single definition of "worse than", used by the trigger, the procedure and the API's risk ordering.

Stored procedure. `sp_declining_repos(p_days)` pairs each repository's latest snapshot with the newest snapshot of the same scoring version taken at least `p_days` earlier, and returns repositories whose effective bus factor fell or whose risk level rose.

Trigger. `trg_snapshot_risk_alert` (Listing 2.3) fires after every snapshot insert. It compares the new risk level with the repository's previous snapshot *in time* and of the same scoring version, and writes a row to `risk_alert` when the risk has increased. The alert is dated at the snapshot's own `computed_at`, so alerts produced during the back-fill carry the date the risk actually rose. Because the trigger runs inside the sync transaction, a rolled-back sync can never leave an alert pointing at a snapshot that does not exist.

Listing 2.3: Trigger recording risk increases.

```
CREATE TRIGGER trg_snapshot_risk_alert
AFTER INSERT ON bus_factor_snapshot
FOR EACH ROW
BEGIN
    DECLARE v_prev_level VARCHAR(10);
    SET v_prev_level = (
        SELECT s.risk_level FROM bus_factor_snapshot s
        WHERE s.repo_id = NEW.repo_id
          AND s.scoring_version = NEW.scoring_version
          AND s.snapshot_id <> NEW.snapshot_id
          AND s.computed_at < NEW.computed_at
        ORDER BY s.computed_at DESC, s.snapshot_id DESC
        LIMIT 1);
    IF v_prev_level IS NOT NULL
       AND fn_risk_rank(NEW.risk_level) > fn_risk_rank(v_prev_level) THEN
        INSERT INTO risk_alert (repo_id, snapshot_id, previous_level, new_level, created_at)
        VALUES (NEW.repo_id, NEW.snapshot_id, v_prev_level, NEW.risk_level, NEW.computed_at);
    END IF;
END
```

2.9 Application layer

REST API. A FastAPI [18] service exposes nine endpoints: a risk-ranked repository list with filtering and sorting, repository detail with contributor breakdown (bots marked, last-12-months commits) and risk reasons, snapshot history, dashboard summary counts, declining repositories (via the stored procedure), recent alerts, GitHub search, and adding or removing a tracked repository. The trend endpoint keeps one point per day, the day's latest snapshot, using `ROW_NUMBER()` partitioned by date. Database connections come from a pool of five; every query is parameterised.

Dashboard. A React [19] single-page application (Figure 3.4) shows summary counts per risk level, the recent risk increases, and a searchable, filterable risk-ranked table. The detail page (Figures 3.5 and 3.3) shows both bus factors, the reasons for the risk level, the burnout signals, the bus-factor trend and the contributor breakdown.

Chapter 3

Results and Discussion

3.1 Experimental setup

The system was run on a Windows 11 laptop with MySQL Community Server 8.0.46, Python 3.12 and Node.js. The tracked list contained 43 public repositories: large, well-staffed projects (e.g. React, Kubernetes, CPython, Rust), popular single-maintainer utilities (e.g. `is-number`, `p-limit`, `chalk`), libraries known to be in maintenance mode or deprecated (`request`, `moment`) and one archived project (`jquery-mobile`). All figures below use each repository's latest snapshot (scoring version 2) from the sync of 17 October 2026, and the 12 monthly snapshots back-filled from November 2025 to October 2026.

3.2 Data collected

One repository (`substack/node-mkdirp`) no longer exists on GitHub and returned HTTP 404; the remaining 42 were synchronised. Table 3.1 lists the resulting row counts; the database occupies 44 MB including indexes.

Table 3.1: Rows stored per relation.

Relation	Rows	Relation	Rows
repository	42	pull_request	16,490
contributor (52 bots)	17,306	repo_release	3,145
repo_contributor	10,053	weekly_activity	2,549
contributor_weekly_commits	273,603	bus_factor_snapshot	833
issue	6,211	risk_alert	25

Of the 833 snapshots, 631 were produced by scoring version 2 (547 of them by the back-fill); 16 of the 25 alerts belong to version 2.

Correcting the response times. The first version of the sync took the very first comment on an issue as its response. When the bot-aware rule was introduced, a maintenance script re-fetched the 1,043 stored response times that could still be looked up. 217 changed, including

89 issues whose only comments came from bots or the issue's author. The 467 that could no longer be looked up were cleared rather than kept, leaving 954 verified response times.

3.3 Risk distribution

Figure 3.1 and Table 3.2 summarise the risk levels. **21 of the 42 repositories (50%) have an effective bus factor of 1**: one person wrote at least half of their recent commits (or, for the seven repositories with fewer than 10 recent commits, of all commits). After the burnout signals are applied, 27 repositories are Critical, 2 High, 2 Medium and 11 Healthy.

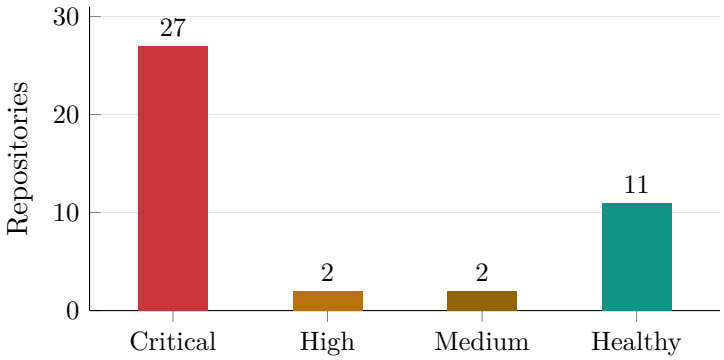


Figure 3.1: Risk level of the 42 repositories (latest snapshot).

Table 3.2: Risk levels with average effective bus factor and top-contributor share.

Risk level	Repositories	Avg. bus factor	Avg. top-contributor share
Critical	27	1.3	70.4%
High	2	2.0	44.7%
Medium	2	4.0	24.5%
Healthy	11	12.4	10.7%

Figure 3.2 plots each repository's effective bus factor against its top contributor's share of commits in the same window. Every repository whose top contributor wrote more than half of the commits has a bus factor of 1 by definition, and every Healthy repository has a top contributor below 25%. Popularity offers no protection: `scrcpy` (over 150,000 stars) has a bus factor of 1, with 89.7% of last year's commits from one person, and `flask` (75,000 stars) has 96.2%.

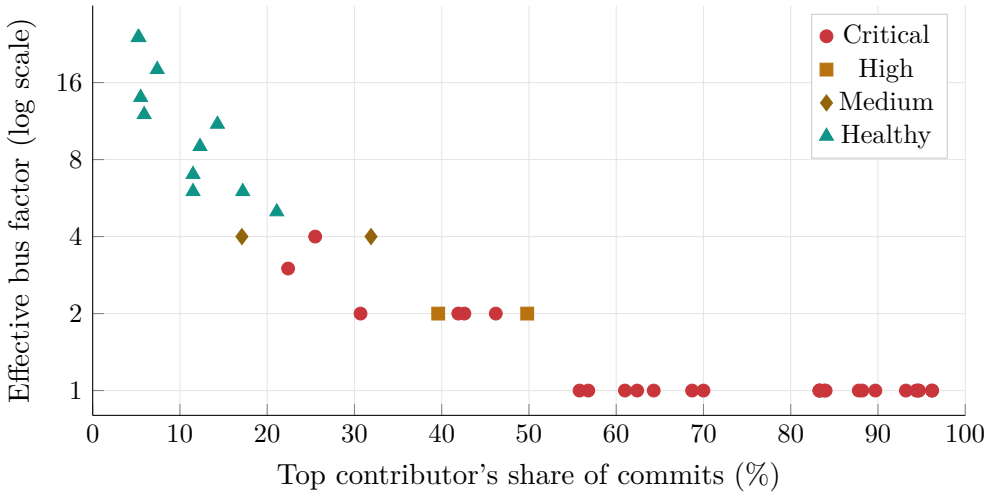


Figure 3.2: Effective bus factor versus top-contributor share for the 42 repositories.

Concentration is also uneven across ecosystems (Table 3.3). The 20 JavaScript repositories have an average effective bus factor of 2.0, against 5.4 for the 8 Python repositories, which reflects the many small single-purpose npm packages in the sample. The sample per language is small and was not drawn randomly, so this is an observation about this data set, not a general claim.

Table 3.3: Average effective bus factor by primary language (languages with ≥ 2 repositories).

Language	Repositories	Avg. bus factor
JavaScript	20	2.0
Python	8	5.4
TypeScript	5	4.2
Go	2	16.0
C	2	2.5

3.4 Effect of excluding bots and measuring recent work

Bots. Excluding the 52 bot accounts changes large projects most. Kubernetes' all-time bus factor rises from 18 to 44 once `k8s-ci-robot` (27,260 commits) is excluded, and TensorFlow's from 18 to 76 without `tensorflower-gardener` (59,810 commits). Both are Healthy either way, but their reported top contributor is now a person.

Recency. For 35 of the 42 repositories the rating is based on the last 12 months; the other seven had fewer than 10 recent commits. Measuring recent work corrects cases in both directions:

- **Express** has an all-time bus factor of 1 (its original author wrote most of its history) but a recent bus factor of 2: it is rated High, not Critical.
- **Flask** goes the other way: its all-time bus factor is 2, but one maintainer wrote 96.2% of last year's commits, a recent bus factor of 1.
- **jQuery** (all-time 4, recent 1) and **requests** (all-time 3, recent 1) likewise now depend on a single active maintainer.

Burnout signals. Across the 42 repositories, the key-contributor inactivity signal fired for 6, a commit-activity fall of 50% or more for 4, and "no commits in the last 12 months" for 4 (`is-number`, `request`, `jquery-mobile`, `gulp`). One repository is archived. With bot replies removed, no repository's median issue response exceeded 30 days.

3.5 Trends and alerts

The back-filled history makes the trend features usable immediately. The call

```
CALL sp_declining_repos(180);
```

returns 12 repositories whose effective bus factor fell or whose risk rose over the last six months. The clearest case, `httpx`, is shown in Figure 3.3: its bus factor was between 4 and 5 from December 2025 to August 2026, and has been 1 since September. Recent activity fell below 10 commits a year, so the rating now rests on the all-time bus factor, which is dominated by its original author, who has not committed for 398 days.



Figure 3.3: Bus factor over time for `encode/httpx`: monthly back-filled snapshots from November 2025 and the current one.

The trigger recorded 16 risk increases in the version-2 history (Table 3.4), each dated at the snapshot where the risk rose.

Table 3.4: Risk increases recorded by `trg_snapshot_risk_alert` (scoring version 2).

Date	Repository	Before	After
2025-12-01	gulpjs/gulp	Medium	High
2026-01-01	psf/requests	High	Critical
2026-02-01	facebook/react	Medium	High
2026-03-01	jashkenas/underscore	High	Critical
2026-03-01	expressjs/express	Medium	High
2026-04-01	encode/httpx	Healthy	Medium
2026-05-01	facebook/react	Medium	High
2026-05-01	expressjs/express	High	Critical
2026-06-01	gulpjs/gulp	High	Critical
2026-06-01	encode/httpx	Medium	High
2026-07-01	django/django	Healthy	Medium
2026-07-01	sindresorhus/execa	High	Critical
2026-09-01	encode/httpx	High	Critical
2026-10-01	lodash/lodash	High	Critical
2026-10-17	pypa/pip	High	Critical
2026-10-17	jquery/jquery-mobile	High	Critical

Two caveats apply. Alerts that were followed by a recovery remain in the log, which is intended: React rose to High twice and is Healthy today. And `jquery-mobile`'s alert on 17 October reflects the archived flag, which is only known for today, not a change on that day.

3.6 Case studies and validation

Figure 3.5 shows the explanation produced for `httpx`. Several cases act as sanity checks on the method:

- **Known-healthy projects score Healthy.** React, Node.js, Rust, PyTorch, CPython, Kubernetes, Go and VS Code all have effective bus factors of 5 or more, with no contributor above 22% of recent commits.
- **Known-unmaintained projects score badly.** The archived `jquery-mobile`, the deprecated `request` and the maintenance-mode `moment` are all Critical.
- **Calibration of the inactivity signal.** In a first version the inactivity signal fired whenever the top contributor had been silent for 180 days. This rated React as Medium because its all-time top contributor, with only 10.2% of commits, had not committed for 264 days. Such turnover is routine in a large project, so the signal now requires the inactive person to hold at least 25% of commits.



Figure 3.4: Dashboard: summary counts per risk level and the recent risk increases recorded by the trigger, dated by when the risk rose.

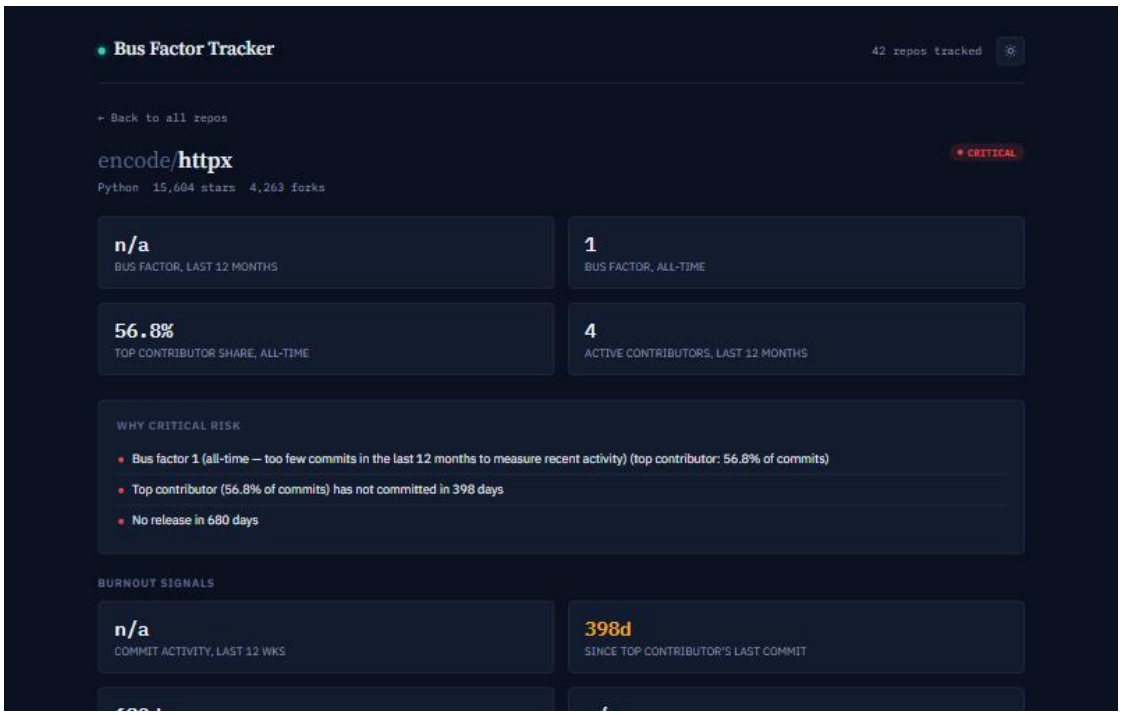


Figure 3.5: Repository detail for `encode/httpx`: recent and all-time bus factor, the reasons for the risk level, and the burnout signals.

3.7 Performance

Sync. The full sync of 43 repositories with scoring version 2 took 1,267.9 s (about 29 s per repository). It wrote 273,603 weekly commit rows and back-filled 547 snapshots, and the time is

dominated by GitHub API latency. During the run the engine recovered from repeated dropped and truncated connections without losing any repository.

API. Table 3.5 shows the median latency of each read endpoint over 20 requests, measured in-process against the local database. The repository-detail endpoint originally took 186 ms for React because its commit-share column used a correlated subquery that re-summed the repository's commits for every one of its 504 contributor rows. Replacing it with the window function `SUM(...) OVER ()` computes the total once. Even with the per-contributor last-12-months join added later, the endpoint takes 10.4 ms.

Table 3.5: Median API latency over 20 requests.

Endpoint	Median latency
`GET /api/repos` (risk-ranked list, via view)	6.0 ms
`GET /api/repos/facebook/react` (correlated subquery, v1)	186.3 ms
`GET /api/repos/facebook/react` (window function)	10.4 ms
`GET /api/repos/{owner}/{name}/trend`	4.0 ms
`GET /api/stats/summary`	4.9 ms
`GET /api/stats/declining` (stored procedure)	6.6 ms
`GET /api/alerts`	4.0 ms

3.8 Per-repository results

Table 3.6 lists the latest snapshot for every repository.

Table 3.6: Latest snapshot per repository (bots excluded). BF 12m = bus factor over the last 12 months ("–" = fewer than 10 recent commits); BF all = all-time bus factor; Top = top contributor's share of commits in the window the rating uses; ΔAct = commit-activity change ("–" = baseline below one commit per week); Inact. = days since that contributor's last commit.

Repository	Lang.	BF 12m	BF all	Top %	ΔAct %	Inact.	Risk
Genymobile/scrcpy	C	1	1	89.7	7.3	13	Critical
laravel/laravel	Blade	1	1	61.0	−61.4	13	Critical
pallets/flask	Python	1	2	96.2	8.3	13	Critical
jquery/jquery	JS	1	4	64.3	−27.5	13	Critical
vuejs/core	TS	1	1	62.4	−12.4	6	Critical
psf/requests	Python	1	3	94.7	−94.0	55	Critical
moment/moment	JS	1	2	87.8	–	34	Critical
tj/commander.js	JS	1	1	84.0	–	146	Critical
Automattic/mongoose	JS	1	1	70.0	−20.5	6	Critical
jashkenas/underscore	JS	1	3	83.8	–	6	Critical
chalk/chalk	JS	1	1	83.3	–	20	Critical
motdotla/dotenv	JS	1	1	88.2	78.7	6	Critical
sindresorhus/ky	TS	1	1	93.2	290.2	13	Critical
BtbN/FFmpeg-Builds	Shell	1	1	83.3	145.2	20	Critical
benoitc/gunicorn	Python	1	1	94.4	−1.3	41	Critical
cordiverse/cordis	TS	1	1	68.7	386.3	6	Critical

Repository	Lang.	BF 12m	BF all	Top %	ΔAct %	Inact.	Risk
sindresorhus/execa	JS	1	1	94.6	–	6	Critical
sindresorhus/p-limit	JS	1	1	83.3	–	34	Critical
encode/httpx	Python	–	1	56.8	–	398	Critical
sindresorhus/is-online	JS	–	1	55.8	–	34	Critical
jonschlinkert/is-number	JS	–	1	96.2	–	3,030	Critical
lodash/lodash	JS	2	1	42.6	−85.2	181	Critical
pypa/pip	Python	2	6	46.2	−51.4	6	Critical
caolan/async	JS	–	2	41.9	–	776	Critical
request/request	JS	–	2	30.7	–	2,442	Critical
jquery/jquery-mobile†	JS	–	3	22.4	–	3,702	Critical
gulpjs/gulp	JS	–	4	25.5	–	503	Critical
expressjs/express	JS	2	1	39.6	–	97	High
mochajs/mocha	JS	2	3	49.8	0.7	13	High
django/django	Python	4	8	31.9	−31.9	6	Medium
FFmpeg/FFmpeg	C	4	5	17.1	20.5	48	Medium
facebook/react	JS	5	6	21.1	−17.6	6	Healthy
angular/angular	TS	6	14	11.5	−22.1	6	Healthy
rails/rails	Ruby	6	13	17.2	39.7	6	Healthy
nodejs/node	JS	7	13	11.5	117.0	6	Healthy
python/cpython	Python	9	14	12.3	−1.2	6	Healthy
rust-lang/rust	Rust	11	20	14.3	17.9	6	Healthy
microsoft/vscode	TS	12	7	5.9	−8.6	6	Healthy
golang/go	Go	14	12	5.5	15.3	13	Healthy
kubernetes/kubernetes	Go	18	44	7.4	−41.0	6	Healthy
tensorflow/tensorflow	C++	24	76	5.3	9.8	13	Healthy
pytorch/pytorch	Python	24	49	5.2	9.1	6	Healthy

†Archived by its owner.

3.9 Discussion of limitations

The second version of the method removes the three main weaknesses found in the first: bots counted as contributors, an all-time bus factor that over-weighted people who left long ago, and the absence of history before the first sync. The following limitations remain:

- **Commits are a proxy for effort.** Code review, issue triage, releases and documentation are not counted, and in projects where committers apply other people's patches (as in Django) commits credit the person who merges.
- **Bot detection is name-based.** A bot with an ordinary-looking name is not caught. Go's median issue response is still under an hour, which may come from such an account.
- **Contributor statistics are capped.** GitHub's statistics cover each repository's top 100 contributors, so the long tail of very large projects is incomplete. Commits from e-mail addresses not linked to a GitHub account are missing altogether.
- **Switching to the all-time bus factor can cause a jump.** When a repository's recent

activity falls below 10 commits a year, the rating moves from the recent to the all-time bus factor, as for `httpx` in September 2026. The reasons list states this explicitly.

- **Some history cannot be reconstructed.** The archived flag is only known for today, and response times are sparse for older issues, so back-filled snapshots rely mainly on commit data.
- **The sample is small and hand-picked.** 42 repositories chosen to cover healthy, small and abandoned projects are enough to test the method, not to draw ecosystem-wide conclusions.

3.10 Challenges and lessons learned

- **A query that silently returned nothing.** The first trend query joined snapshots on `s2.computed_at = s1.computed_at - INTERVAL 6 MONTH`. Snapshots are taken whenever the sync runs, so no two are exactly six months apart to the second, and the query always returned zero rows. It now selects the newest snapshot *at or before* the cut-off. Lesson: test every query on real data, not just for syntax.
- **Bots in the data.** Real data showed CI bots ranked as the top contributors of Kubernetes, TensorFlow and PyTorch, and bots replying to every new issue within seconds. Both distort the measurements and are now filtered out.
- **Changing the method without corrupting the history.** The first re-score under the new method would have looked like a wave of real risk changes. Versioned snapshots, compared only within a version, keep method changes and project changes apart.
- **Driver behaviour.** The installed MySQL connector returns stored-procedure results as dictionaries, which broke the API's tuple-based handling only once real rows existed. The endpoint now accepts both forms.
- **Porting from PostgreSQL to MySQL.** The design was first written for PostgreSQL. MySQL needed a different `INTERVAL` syntax, a window-function median instead of `PERCENTILE_CONT`, and a renamed `release` table.
- **Calibrating on real data.** The React case showed that a signal that looks reasonable in isolation can mislabel large projects; thresholds must be checked against known cases.

Chapter 4

Conclusion and Future Work

4.1 Conclusion

This project built a working, database-centred system that estimates the bus factor and maintainer-burnout risk of real open-source repositories using only public GitHub data. The design applies core DBMS concepts where they solve real problems:

- a normalised schema in BCNF with cascading referential integrity;
- one ACID transaction per repository, so a failed sync never corrupts the current state;
- CTEs and window functions that compute every measurement inside the database, for any time window and as of any past date;
- a view, stored function, stored procedure and trigger that expose current risk, detect declines and record risk increases;
- versioned snapshots, so the method can change without corrupting the history.

On 42 widely used repositories the system showed that the risk is widespread: half of them depend on one person for at least half of their recent commits, including projects with tens of thousands of stars. A year of history rebuilt from weekly commit data revealed 16 risk increases and declines such as `httpx`'s fall from a bus factor of 5 to 1. Every risk level is explained in plain English, and the API answers in about 10 ms.

4.2 Future work

- Count review, triage and release work, not only commits.
- Read commit history from cloned repositories to include contributors beyond GitHub's top 100 and commits from unlinked e-mail addresses.
- Analyse a project's full dependency tree to report the riskiest dependencies of an application.
- Send e-mail or webhook notifications when a tracked repository becomes Critical.
- Validate the risk levels against a labelled set of projects that were later abandoned.

References

[1] Nadia Eghbal. *Roads and Bridges: The Unseen Labor Behind Our Digital Infrastructure*. Ford Foundation, 2016.

[2] Nadia Eghbal. *Working in Public: The Making and Maintenance of Open Source Software*. Stripe Press, 2020.

[3] npm, Inc. kik, left-pad, and npm. npm Blog, `https://blog.npmjs.org/post/141577284765/kik-left-pad-and-npm`, 2016. Published 23 March 2016.

[4] Jailton Coelho and Marco Tulio Valente. Why modern open source projects fail. In *Proceedings of the 11th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*, pages 186–196, 2017.

[5] Naveen Raman, Minxuan Cao, Yulia Tsvetkov, Christian Kästner, and Bogdan Vasilescu. Stress and burnout in open source: Toward finding, understanding, and mitigating unhealthy interactions. In *Proceedings of the 42nd International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER)*, pages 57–60, 2020.

[6] Guilherme Avelino, Leonardo Passos, Andre Hora, and Marco Tulio Valente. A novel approach for estimating truck factors. In *Proceedings of the 24th IEEE International Conference on Program Comprehension (ICPC)*, pages 1–10, 2016.

[7] Valerio Cosentino, Javier Luis Cánovas Izquierdo, and Jordi Cabot. Assessing the bus factor of Git repositories. In *Proceedings of the 22nd IEEE International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, pages 499–503, 2015.

[8] Elgun Jabrayilzade, Mikhail Evtikhiev, Eray Tüzün, and Vladimir Kovalenko. Bus factor in practice. In *Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 97–106, 2022.

[9] CHAOSS Project. Community health analytics in open source software. `https://chaoss.community/`, 2026. Accessed October 2026.

[10] Open Source Security Foundation. OpenSSF scorecard. `https://github.com/ossf/scorecard`, 2026. Accessed October 2026.

[11] GitHub, Inc. GitHub REST API documentation. `https://docs.github.com/en/rest`, 2026. Accessed October 2026.

[12] Peter Pin-Shan Chen. The entity-relationship model—toward a unified view of data. *ACM Transactions on Database Systems*, 1(1):9–36, 1976.

[13] Edgar F. Codd. A relational model of data for large shared data banks. *Communications of the ACM*, 13(6):377–387, 1970.

[14] Abraham Silberschatz, Henry F. Korth, and S. Sudarshan. *Database System Concepts*. McGraw-Hill, 7th edition, 2019.

[15] Ramez Elmasri and Shamkant B. Navathe. *Fundamentals of Database Systems*. Pearson, 7th edition, 2016.

[16] Theo Haerder and Andreas Reuter. Principles of transaction-oriented database recovery. *ACM Computing Surveys*, 15(4):287–317, 1983.

[17] Oracle Corporation. MySQL 8.0 reference manual. `https://dev.mysql.com/doc/refman/8.0/en/`, 2026. Accessed October 2026.

[18] Sebastián Ramírez. FastAPI documentation. `https://fastapi.tiangolo.com/`, 2026. Accessed October 2026.

[19] Meta Platforms, Inc. React documentation. `https://react.dev/`, 2026. Accessed October 2026.